

Семинар по C++ №9

Перегрузка операторов

Конструктор

- `ComplexNumber() = default;` - принудительное создание конструктора по умолчанию
- Поля инициализируется в порядке объявления в самом классе
- **Rule of Three** – copy constructor, destructor, copy assignment operator

Оператор присваивания

Присваивание самому себе:

`*this == other` – сравнение объектов

`this == &other` – сравнение указателей

Идиома [copy-and-swap](#):

1. Принимаем по значению (конструктор копированием)
2. Делаем swap полей

Операторы +, +=

- Выражаем + через += - экономим на копировании
- + не надо делать членом класса, ибо левый операнд может не быть объектом класса
- Нужно писать `a += b; return a;` вместо `return a+=b;` (RVO)
- Что делать с `a + b = 5`?
 - Возвращать `const`
 - Докинуть `&` в оператор присваивания справа

Сравнения

- Обычно реализуют оператор `<`
- Остальные выражаются через `<`
- Не пишите `a > b` как `(a != b) && !(a < b)`
- Лучше пишите `b < a`
- Старайтесь избегать друзей (friend)

Spaceship operator ($\leq$)

- [Беглый взгляд](#)
- [Подробная статья](#)
- [std::strong_ordering, std::weak_ordering или std::partial_ordering?](#)
- [Насколько полезно?](#)

<=>: идея

Хотим упростить
перегрузку
операторов

Без оператора spaceship	С оператором spaceship
<pre>struct point3d { int x; int y; int z; }; using p_ref = const point3d&; bool operator==(p_ref a, p_ref b) { return a.x == b.x && a.y == b.y && a.z == b.z; } bool operator< (p_ref a, p_ref b) { return a.x < b.x (a.x == b.x && a.y < b.y) (a.x == b.x && a.y == b.y && a.z < b.z); } bool operator!=(p_ref a, p_ref b) { return !(a == b); } bool operator<=(p_ref a, p_ref b) { return !(b < a); } bool operator> (p_ref a, p_ref b) { return b < a; } bool operator>=(p_ref a, p_ref b) { return !(a < b); }</pre>	<pre>struct point3d { int x; int y; int z; auto operator<=>(const point3d&) const = default; };</pre>

$\leq$: orderings

Правда ли, что:

a) $x == y \Leftrightarrow !(x < y) \ \&\& \ !(y < x)$

b) $x == y \Leftrightarrow x$ неотличимо от y

“Неотличимо” – с помощью `const public` полей и методов класса

$\Leftrightarrow$: orderings

Правда ли, что:

а) $x == y \Leftrightarrow !(x < y) \ \&\& \ !(y < x)$ - только в ЛУМ.

Пример: float

а) $x == y \Leftrightarrow x$ неотлично от y - не всегда.

Пример: строки; точки на плоскости при сравнении только по одной координате

“==” - equivalence

“неотлично” - equality

$\leq$: orderings

std::strong_ordering – ЛУМ с equality

std::weak_ordering - ЛУМ

std::partial_ordering - ЧУМ

std::strong_ordering

std::strong_ordering::less

std::strong_ordering::greater

std::strong_ordering::equal

std::strong_ordering::equivalent

std::weak_ordering

std::weak_ordering::less

std::weak_ordering::greater

std::weak_ordering::equivalent

(Не существует std::weak_ordering::equal)

`std::partial_ordering`

`std::partial_ordering::less`

`std::partial_ordering::greater`

`std::partial_ordering::equivalent`

`std::partial_ordering::unordered`

(Не существует `std::partial_ordering::equal`, зато есть `unordered`)

Сравнения

- Умеем выражать все сравнения через `<=>` и `==`:

```
strong_ordering::less < 0    // true
strong_ordering::less == 0   // false
strong_ordering::less != 0   // true
strong_ordering::greater >= 0 // true

partial_ordering::less < 0    // true
partial_ordering::greater > 0 // true

// unordered - особое значение, которое невозможно
// сравнить ни с каким другим значением
partial_ordering::unordered < 0 // false
partial_ordering::unordered == 0 // false
partial_ordering::unordered > 0 // false
```

- [Пример](#) (совет: перейти в `cpp insights`)

Базовые и производные операторы

	Равенство	Упорядочение
Базовые	==	<=>
Производные	!=	<, >, <=, >=

Правила:

- Обращение доступно только для базовых операторов
- Переписываться могут только производные операторы (через соответствующие базовые)
- При поиске кандидатов за один проход ищутся все операторы с данным именем, а также все их обращённые и переписанные версии
- Если наилучший кандидат является переписанной или обращённой версией и при этом такая замена является недопустимой, программа считается некорректной.

Поиск кандидата

Исходная операция	Вариант 1	Вариант 2
$a == b$	$b == a$	
$a != b$	$!(a == b)$	$!(b == a)$
$a <=> b$	$0 <=> (b <=> a)$	
$a < b$	$(a <=> b) < 0$	$(b <=> a) > 0$
$a <= b$	$(a <=> b) <= 0$	$(b <=> a) >= 0$
$a > b$	$(a <=> b) > 0$	$(b <=> a) < 0$
$a >= b$	$(a <=> b) >= 0$	$(b <=> a) <= 0$

- Столбец слева имеет больший приоритет чем столбец справа
- Второй и третий столбец содержат только базовые операторы

Бонус

- Можно попросить сгенерировать оператор `<=>`, тогда сравнение будет лексикографическое
- Оператор `==` может быть тоже неявно сгенерирован
- Благодаря обращению операторы сравнения могут быть функциями-членами класса

Вывод в поток

- В классе объявляем friend
- Снаружи пишем:
- `std::ostream& operator<<(std::ostream& os, const String& s) {`
- `...`
- `return os << ... ;`



Мне кажется, я что-то забыл.

Если ты забыл, значит
это было не важно.



Да, ты прав.

CONST

const

Для того, чтобы перегрузка операторов корректно работала с константными объектами, добавляйте `const`.

Если вы не можете добавить `const`, перегрузите оператор и добавьте `const`.

`const` это важно.